DBMS & PostgreSQL

65 Interview Questions & Answers

In Simple, Easy Language — From Basics to Advanced

✓ DBMS Core Concepts (ACID, Transactions, Normalization)
✓ SQL Fundamentals (Joins, Aggregates, Subqueries)
✓ Indexing, Keys, Constraints & Views
✓ Concurrency, Locking & Deadlocks
✓ PostgreSQL Specific — MVCC, JSONB, Partitioning
✓ PostgreSQL Performance — EXPLAIN, pg_stat, Vacuuming
✓ PostgreSQL Advanced — WAL, Replication, CTEs, Window Functions

How to Use This Guide

Blue questions (Q1–Q40) cover general DBMS concepts that every developer must know — these are asked in almost every database interview regardless of which database you use.

Green questions (Q41–Q65) are PostgreSQL-specific — marked with a green header. These are asked when the job involves PostgreSQL specifically. They cover things that make PostgreSQL unique: MVCC, JSONB, table partitioning, WAL, replication, and performance tuning.

All answers are written in simple, everyday language first — then backed up with SQL examples where helpful. No jargon without explanation.

Table of Contents

1. Section 1: DBMS Basics & Core Concepts (Q1–Q10)
2. Section 2: SQL Fundamentals — Keys, Constraints, Joins (Q11–Q20)
3. Section 3: SQL Queries — Aggregates, Subqueries, Views (Q21–Q30)
4. Section 4: Normalization, Transactions & Concurrency (Q31–Q40)
5. Section 5: PostgreSQL Fundamentals (Q41–Q48) 🐘
6. Section 6: PostgreSQL Indexing & Performance (Q49–Q55) 🐘
7. Section 7: PostgreSQL Advanced Features (Q56–Q65) 🐘

Legend

Question Background	Header Colour	Who Asks This?
DBMS / General SQL — applies to any database	Blue header	Every database interview
PostgreSQL-specific — unique to Postgres	Green header	Jobs mentioning PostgreSQL

Section 1: DBMS Basics & Core Concepts (Q1–Q10)

These questions check your understanding of what a database is and why we use one. Think of them as the 'building blocks' questions — almost every interview starts here.

Q1: What is a Database and what is a DBMS? Give a real-world example.

Answer: A database is an organised collection of data stored so it can be easily found and used. Think of it like a digital filing cabinet. A DBMS (Database Management System) is the software that manages that cabinet — it lets you add, find, update, and delete data safely. Real-world example: When you order food on Swiggy, your order details, address, and payment are stored in their database, managed by a DBMS like PostgreSQL or MySQL.

Q2: What is the difference between DBMS and RDBMS?

Answer: DBMS stores data as files and does not necessarily link data together. RDBMS (Relational DBMS) stores data in tables (rows and columns), and tables can be connected to each other through relationships using keys. RDBMS also enforces data integrity rules. Examples: DBMS — Microsoft Access, XML databases. RDBMS — PostgreSQL, MySQL, Oracle. PostgreSQL is an Object-Relational DBMS (ORDBMS) — a superset of RDBMS that also supports custom data types and object-like features.

Q3: What is the difference between a database schema and a database instance?

Answer: Think of a schema as the blueprint or structure — it defines the table names, column names, data types, and relationships. An instance is the actual data in the database at a particular point in time. The schema rarely changes; the instance changes every time you insert, update, or delete data. Analogy: Schema = the design of a house (rooms, layout). Instance = the actual furniture and people inside the house at this moment.

Q4: What are the three levels of data abstraction in DBMS?

Answer: DBMS hides complexity from users using three levels: 1) Physical Level (Internal) — how data is actually stored on disk (files, blocks, indexes). Users never see this. 2) Logical Level (Conceptual) — what data is stored and the relationships between them (tables, columns, foreign keys). This is what developers work with. 3) View Level (External) — what individual users or applications see (often a subset via views or queries). Each user sees only what they need.

Q5: What is Data Independence and why is it important?

Answer: Data independence means you can change one level of the database (physical or logical) without affecting the other levels or the applications that use it. Two types: Physical Data Independence — change how data is stored on disk without changing the table structure or application code. Logical Data Independence — change table structure (add a column) without changing the application views or queries that use it. It's important because databases evolve over time, and you don't want every application to break every time you restructure storage.

Q6: What is a Primary Key, Foreign Key, and Composite Key? Explain with examples.

Answer: Primary Key uniquely identifies each row in a table. No duplicates, no NULLs allowed. Example: employee_id in an Employee table. Foreign Key is a column in one table that refers to the Primary Key of another table — creating a link between them. Example: department_id in Employee table links to id in Department table. Composite Key is a Primary Key made of two or more columns together. Example: (student_id, course_id) together form the primary key of an Enrollment table — neither alone is unique, but both together are.

```
-- Employee table: primary key
CREATE TABLE employee (
    employee_id SERIAL PRIMARY KEY,
    name        VARCHAR(100),
```

```
    department_id INT REFERENCES department(id)  -- foreign key
);

-- Enrollment table: composite primary key
CREATE TABLE enrollment (
    student_id INT,
    course_id  INT,
    PRIMARY KEY (student_id, course_id)
);
```

Q7: What are the different types of keys in DBMS?

Answer: Super Key: Any set of columns that can uniquely identify a row (may have extra columns). Candidate Key: The minimal super key — no extra columns. A table can have multiple candidate keys. Primary Key: The chosen candidate key for the table. Alternate Key: Any candidate key that was NOT chosen as the primary key. Foreign Key: Links one table to another through a primary key reference. Surrogate Key: An artificial, system-generated key (like SERIAL/auto-increment) with no business meaning — purely for uniqueness.

Q8: What is an Entity-Relationship (ER) Diagram and what are its components?

Answer: An ER Diagram is a visual blueprint showing the tables (entities) in a database and how they are related. Entities are the main objects (rectangle shapes) — like Customer, Order, Product. Attributes are the properties of entities (oval shapes) — like customer name, product price. Relationships show how entities are connected (diamond shapes) — like 'Customer PLACES Order'. Cardinality shows how many: One-to-One (1:1), One-to-Many (1:M), or Many-to-Many (M:N). ER diagrams are drawn before writing any SQL — they are the database design plan.

Q9: What are database constraints? List and explain all types.

Answer: Constraints are rules that protect data quality. NOT NULL — a column must always have a value (cannot be empty). UNIQUE — no two rows can have the same value in this column. PRIMARY KEY — combines NOT NULL + UNIQUE, identifies each row. FOREIGN KEY — value must match a valid value in another table's primary key. CHECK — value must satisfy a condition, e.g. age >= 18. DEFAULT — if no value is given, use this default value.

```
CREATE TABLE product (
    id          SERIAL PRIMARY KEY,
    name        VARCHAR(100) NOT NULL,
    email       VARCHAR(150) UNIQUE,
    price       NUMERIC CHECK (price > 0),
    category    VARCHAR(50) DEFAULT 'General'
);
```

Q10: What is the difference between DROP, DELETE, and TRUNCATE?

Answer: All three remove data, but in very different ways. DELETE removes specific rows that match a WHERE condition. It is slow (row-by-row), logs each deleted row, can be rolled back, and fires triggers. TRUNCATE removes ALL rows in a table instantly. It is very fast, does minimal logging, and cannot use WHERE. In PostgreSQL, TRUNCATE CAN be rolled back inside a transaction. DROP removes the entire table (structure + data) permanently. Think of it as: DELETE = remove selected pages from a book. TRUNCATE = erase all pages but keep the book cover. DROP = throw the entire book in the bin.

Section 2: SQL Fundamentals — Keys, Constraints, Joins (Q11–Q20)

These questions check your SQL fluency — how you link tables, filter data, and understand what SQL commands actually do under the hood.

Q11: What are the different types of SQL commands? Explain DDL, DML, DCL, TCL.

Answer: SQL commands are grouped into four categories. DDL (Data Definition Language) defines the database structure: CREATE, ALTER, DROP, TRUNCATE, RENAME. DML (Data Manipulation Language) manipulates the data: SELECT, INSERT, UPDATE, DELETE. DCL (Data Control Language) controls access and permissions: GRANT, REVOKE. TCL (Transaction Control Language) manages transactions: COMMIT, ROLLBACK, SAVEPOINT. Most interviews ask you to categorise commands like 'Is TRUNCATE DDL or DML?' — it is DDL because it changes the structure (resets the table).

Q12: What are the different types of SQL JOINs? Explain each with a simple example.

Answer: INNER JOIN — returns only rows that have a match in BOTH tables. If a customer has no orders, they are excluded. LEFT JOIN — returns ALL rows from the left table, plus matching rows from the right. Customers with no orders still appear, with NULL for order columns. RIGHT JOIN — same as LEFT JOIN but reversed. FULL OUTER JOIN — returns all rows from both tables, NULLs where there is no match. CROSS JOIN — every row from table A combined with every row from table B (Cartesian product). SELF JOIN — a table joined with itself, useful for hierarchical data like employee-manager relationships.

```
-- INNER JOIN: only customers who have placed at least one order
SELECT c.name, o.total
FROM   customer c
INNER JOIN orders o ON c.id = o.customer_id;

-- LEFT JOIN: all customers, even those with no orders
SELECT c.name, o.total
FROM   customer c
LEFT JOIN orders o ON c.id = o.customer_id;

-- SELF JOIN: employee with their manager's name
SELECT e.name AS employee, m.name AS manager
FROM   employee e
LEFT JOIN employee m ON e.manager_id = m.id;
```

Q13: What is the difference between WHERE and HAVING?

Answer: WHERE filters rows BEFORE grouping. HAVING filters groups AFTER a GROUP BY has been applied. Simple rule: if you are not using GROUP BY, use WHERE. If you are filtering on an aggregate result (like COUNT, SUM, AVG), you must use HAVING.

```
-- WHERE: filter individual rows first
SELECT department, COUNT(*) AS total
FROM   employee
WHERE  salary > 30000         -- filter rows BEFORE grouping
GROUP  BY department
HAVING COUNT(*) > 5;          -- filter groups AFTER grouping
```

Q14: What is the difference between UNION and UNION ALL?

Answer: Both UNION and UNION ALL combine results of two SELECT statements. UNION removes duplicate rows from the combined result — slower because it has to sort/compare to find duplicates. UNION ALL keeps ALL rows including duplicates — faster because no de-duplication step. Use UNION ALL when you know there are no duplicates or you explicitly want them. Use UNION only when you need distinct combined results.

```
-- UNION: removes duplicates
SELECT name FROM students
UNION
SELECT name FROM teachers;

-- UNION ALL: keeps duplicates, faster
SELECT name FROM students
UNION ALL
SELECT name FROM teachers;
```

Q15: What is a subquery? What is the difference between a correlated and non-correlated subquery?

Answer: A subquery is a SELECT query nested inside another query. Non-correlated subquery runs independently — it does not depend on the outer query and is executed once. Correlated subquery references a column from the outer query — it runs once for every row of the outer query (slower but sometimes necessary).

```
-- Non-correlated: runs once, finds average salary
SELECT name FROM employee
WHERE  salary > (SELECT AVG(salary) FROM employee);

-- Correlated: runs for EACH employee row
SELECT name FROM employee e
WHERE  salary > (SELECT AVG(salary) FROM employee
                 WHERE department_id = e.department_id);
```

Q16: What is a VIEW in a database? What are its advantages and limitations?

Answer: A VIEW is a saved SELECT query that behaves like a virtual table. It does not store data — it pulls fresh data from the underlying tables every time you query it. Advantages: simplifies complex queries (write the join once, use it everywhere), provides a security layer (show users only certain columns), logical independence (hide table structure changes). Limitations: standard views cannot be indexed or directly modified in complex cases. Updatable views exist but have restrictions. For performance-heavy use, use a Materialized View instead.

```
-- Create a view for sales above $1000
CREATE VIEW big_sales AS
    SELECT customer_id, order_date, total
    FROM   orders
    WHERE  total > 1000;

-- Use it just like a table
SELECT * FROM big_sales WHERE order_date > '2024-01-01';
```

Q17: What are aggregate functions in SQL? List them with examples.

Answer: Aggregate functions perform a calculation on a set of rows and return a single value. COUNT(*) — count total rows. COUNT(column) — count non-NULL values. SUM(column) — add up all values. AVG(column) — calculate the average. MAX(column) — find the highest value. MIN(column) — find the lowest value. They are almost always used with GROUP BY to calculate per group.

```
SELECT department_id,
       COUNT(*)          AS total_employees,
       AVG(salary)       AS avg_salary,
       MAX(salary)       AS highest_pay,
       MIN(salary)       AS lowest_pay,
       SUM(salary)       AS total_payroll
FROM   employee
GROUP  BY department_id;
```

Q18: What are indexes and why are they used? What are the trade-offs?

Answer: An index is like the index at the back of a textbook — instead of reading every page (full table scan), you jump directly to the right page. Indexes speed up SELECT queries with WHERE, JOIN, and ORDER BY. Trade-offs: indexes make SELECT faster but INSERT, UPDATE, DELETE

slower (because the index must also be updated). Indexes take up extra disk space. Create indexes on columns that you frequently filter or join on. Avoid indexing every column — too many indexes can slow down write-heavy tables.

```
-- Create an index on email column (often searched)
CREATE INDEX idx_user_email ON users(email);

-- Composite index for queries that filter on both columns
CREATE INDEX idx_order_date_status ON orders(order_date, status);

-- Unique index (also enforces uniqueness)
CREATE UNIQUE INDEX idx_product_sku ON products(sku);
```

Q19: What is the difference between a clustered and non-clustered index?

Answer: A clustered index physically reorders the rows of the table on disk to match the index order. There can be only ONE clustered index per table (because data can only be physically sorted one way). In PostgreSQL, the PRIMARY KEY creates a unique B-tree index but not technically a 'clustered index' in the SQL Server sense — though you can use CLUSTER command. A non-clustered index is a separate structure that holds a pointer back to the actual row. You can have many non-clustered indexes. Most PostgreSQL indexes are non-clustered by default.

Q20: What is referential integrity and how is it enforced?

Answer: Referential integrity means the data relationships between tables are always valid — a foreign key value must either be NULL or match a valid primary key value in the referenced table. It prevents 'orphan records' — e.g., an order belonging to a customer that doesn't exist. Enforced through FOREIGN KEY constraints with actions: ON DELETE CASCADE (delete child rows automatically when parent is deleted), ON DELETE SET NULL (set child FK to NULL), ON DELETE RESTRICT (block deletion if children exist — default), ON DELETE NO ACTION.

```
CREATE TABLE orders (
    id          SERIAL PRIMARY KEY,
    customer_id INT REFERENCES customers(id)
                ON DELETE CASCADE   -- delete orders if customer deleted
                ON UPDATE CASCADE,  -- update orders if customer id changes
    total       NUMERIC
);
```

Section 3: SQL Queries — Aggregates, Subqueries, Views (Q21–Q30)

These questions test your practical SQL writing skills — the kind of queries you'd write on a day-to-day basis.

Q21: What is a stored procedure and what are its advantages?

Answer: A stored procedure is a saved block of SQL code stored in the database and executed by name. Think of it like a reusable function. Advantages: code reuse (write once, call many times), better performance (pre-compiled by DB), reduced network traffic (one call instead of many SQL statements), security (users call procedures without knowing table structures). In PostgreSQL, these are called Functions (or Procedures from version 11+), written in PL/pgSQL language.

```
-- PostgreSQL function (stored procedure equivalent)
CREATE OR REPLACE FUNCTION get_employee_salary(emp_id INT)
RETURNS NUMERIC AS $$
BEGIN
    RETURN (SELECT salary FROM employee WHERE id = emp_id);
END;
$$ LANGUAGE plpgsql;

-- Call it
SELECT get_employee_salary(42);
```

Q22: What are triggers in a database? Give an example use case.

Answer: A trigger is a piece of code that automatically runs before or after an INSERT, UPDATE, or DELETE operation on a table. You don't call it manually — the database fires it automatically. Use cases: audit logging (record every change to a log table), enforcing business rules that can't be expressed as a constraint, maintaining derived columns automatically, preventing invalid operations.

```
-- Log every salary change to an audit table
CREATE OR REPLACE FUNCTION log_salary_change()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO salary_audit(emp_id, old_salary, new_salary, changed_at)
    VALUES (OLD.id, OLD.salary, NEW.salary, NOW());
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER salary_update_trigger
AFTER UPDATE OF salary ON employee
FOR EACH ROW EXECUTE FUNCTION log_salary_change();
```

Q23: What is the difference between RANK(), DENSE_RANK(), and ROW_NUMBER()?

Answer: All three assign numbers to rows, but handle ties differently. ROW_NUMBER() gives every row a unique sequential number — no ties, always 1,2,3,4. RANK() gives the same number to tied rows, then skips numbers. Example: two people with the same salary both get rank 1, next person gets rank 3 (skips 2). DENSE_RANK() gives the same number to tied rows but does NOT skip. Both tied people get rank 1, next person gets rank 2. Memory trick: DENSE = dense/compact, no gaps.

```
SELECT name, salary,
    ROW_NUMBER()  OVER (ORDER BY salary DESC) AS row_num,
    RANK()        OVER (ORDER BY salary DESC) AS rnk,
    DENSE_RANK()  OVER (ORDER BY salary DESC) AS dense_rnk
FROM employee;

-- Result example:
-- Alice  90000  1  1  1
```

```
-- Bob    90000  2  1  1   <- RANK skips 2, DENSE_RANK does not
-- Carol  75000  3  3  2   <- RANK=3, DENSE_RANK=2
```

Q24: What are window functions and how are they different from GROUP BY?

Answer: Window functions perform calculations across a set of rows related to the current row — without collapsing rows like GROUP BY does. With GROUP BY, you get one row per group. With window functions, you keep all original rows AND add a calculated column. Use cases: running totals, moving averages, rankings, comparing each row to its group average.

```
-- Running total of sales per month
SELECT order_date, amount,
    SUM(amount) OVER (ORDER BY order_date) AS running_total
FROM orders;

-- Each employee's salary vs their department average
SELECT name, department_id, salary,
    AVG(salary) OVER (PARTITION BY department_id) AS dept_avg
FROM employee;
```

Q25: What is a Common Table Expression (CTE) and when do you use it?

Answer: A CTE is a temporary, named result set defined at the start of a query using the WITH keyword. You can think of it as giving a name to a subquery. CTEs make complex queries readable by breaking them into named steps — like naming a variable in programming. CTEs can also be RECURSIVE — they can reference themselves, which is perfect for hierarchical data like org charts or category trees.

```
-- Simple CTE: find average salary first, then use it
WITH avg_salary AS (
    SELECT department_id, AVG(salary) AS avg
    FROM employee
    GROUP BY department_id
)
SELECT e.name, e.salary, a.avg AS dept_average
FROM   employee e
JOIN   avg_salary a ON e.department_id = a.department_id
WHERE  e.salary > a.avg;

-- Recursive CTE: employee hierarchy (boss -> manager -> staff)
WITH RECURSIVE org_chart AS (
    SELECT id, name, manager_id, 1 AS level
    FROM employee WHERE manager_id IS NULL  -- top of tree
    UNION ALL
    SELECT e.id, e.name, e.manager_id, oc.level + 1
    FROM employee e
    JOIN org_chart oc ON e.manager_id = oc.id
)
SELECT * FROM org_chart ORDER BY level, name;
```

Q26: What is the difference between IN, EXISTS, and ANY/ALL in SQL?

Answer: IN checks if a value matches any value in a list or subquery result. Simpler but can be slow for large subqueries. EXISTS checks if a subquery returns at least one row — it stops as soon as it finds the first match, so it can be faster. ANY/ALL compare a value against a set: salary > ANY(list) means salary is greater than at least one value; salary > ALL(list) means salary is greater than every value in the list.

```
-- IN
SELECT name FROM employee WHERE department_id IN (1, 3, 5);

-- EXISTS (often faster for large datasets)
SELECT name FROM customer c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id);

-- ANY: higher salary than at least one person in dept 2
```

```
SELECT name FROM employee
WHERE salary > ANY (SELECT salary FROM employee WHERE department_id = 2);
```

Q27: What is the difference between a materialized view and a regular view?

Answer: A regular view is a saved query — every time you query it, it re-runs the underlying SQL fresh. No data is physically stored. A materialized view physically stores the query results on disk. It is like a cache — very fast to query, but the data may be stale. You must manually REFRESH it to get updated data. Use materialized views for: expensive aggregation queries, reports that run frequently on large tables, data that doesn't need to be real-time.

```
-- Create a materialized view of monthly sales
CREATE MATERIALIZED VIEW monthly_sales AS
    SELECT DATE_TRUNC('month', order_date) AS month,
           SUM(total) AS revenue
    FROM orders
    GROUP BY 1;

-- Refresh it (update the cached data)
REFRESH MATERIALIZED VIEW monthly_sales;

-- Refresh without blocking reads
REFRESH MATERIALIZED VIEW CONCURRENTLY monthly_sales;
```

Q28: What is the difference between COALESCE and NULLIF in SQL?

Answer: COALESCE(value1, value2, ...) returns the first non-NULL value in the list. It's used to replace NULLs with a default. Example: COALESCE(nickname, first_name, 'Unknown') returns the first non-null value. NULLIF(value1, value2) returns NULL if both values are equal; otherwise returns value1. Useful to avoid division-by-zero errors: NULLIF(total_count, 0) — returns NULL instead of 0, preventing divide-by-zero.

```
-- COALESCE: use 'Unknown' if name is NULL
SELECT COALESCE(name, 'Unknown') AS display_name FROM users;

-- NULLIF: safe division (avoid divide-by-zero)
SELECT total_revenue / NULLIF(total_orders, 0) AS avg_order_value
FROM sales_summary;
```

Q29: Write a query to find the second highest salary from a table.

Answer: Classic interview question. Several approaches: use OFFSET/LIMIT, use a subquery with MAX excluding the top salary, or use DENSE_RANK().

```
-- Method 1: OFFSET (simple but only works for Nth)
SELECT DISTINCT salary FROM employee
ORDER BY salary DESC
LIMIT 1 OFFSET 1;

-- Method 2: Subquery (exclude max salary)
SELECT MAX(salary) AS second_highest
FROM employee
WHERE salary < (SELECT MAX(salary) FROM employee);

-- Method 3: DENSE_RANK (generalises to any N)
SELECT salary FROM (
    SELECT salary,
           DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employee
) t
WHERE rnk = 2;
```

Q30: What is the GROUP BY clause and what rules apply to it?

Answer: GROUP BY collapses rows with the same value in a column into a single group, then applies aggregate functions (COUNT, SUM, etc.) per group. The rule: every column in SELECT must either be in the GROUP BY clause OR be inside an aggregate function. You cannot SELECT a column that is not grouped — the database doesn't know which row's value to show.

```sql
-- How many employees per department, with total payroll
SELECT department_id,
       COUNT(*)    AS headcount,
       SUM(salary) AS total_payroll
FROM   employee
GROUP  BY department_id   -- must group by this
HAVING COUNT(*) > 3       -- only show departments with more than 3 people
ORDER  BY total_payroll DESC;
```

Section 4: Normalisation, Transactions & Concurrency (Q31–Q40)

These questions check deeper understanding — how to design tables properly and how databases handle many users writing data simultaneously.

Q31: What is Normalisation? Explain 1NF, 2NF, 3NF, and BCNF with examples.

Answer: Normalisation is the process of organising a database to reduce data repetition and avoid update problems. Think of it as tidying up — splitting big messy tables into smaller, cleaner ones. 1NF (First Normal Form): Every column must have atomic (indivisible) values. No lists or repeating groups in a column. 2NF: Must be in 1NF, and every non-key column must depend on the ENTIRE primary key (not just part of it — relevant when you have a composite key). 3NF: Must be in 2NF, and no non-key column should depend on another non-key column (no transitive dependencies). BCNF: A stricter version of 3NF — every determinant (what determines another column) must be a candidate key.

```
-- 1NF violation: skills stored as comma list in one column
-- BAD: employee(id, name, skills='Java,Python,SQL')
-- GOOD: separate skills table

-- 3NF violation: city determines zip code (transitive dependency)
-- BAD: orders(order_id, customer_id, city, zip_code)
--   zip_code depends on city, not on order_id
-- GOOD: separate city-zip table
```

Q32: What is denormalisation and when would you use it?

Answer: Denormalisation is intentionally adding redundancy back into a normalised database to improve read query performance. Instead of doing complex joins every time, you store pre-joined data. When to use: read-heavy applications where query speed matters more than perfect data consistency, reporting/analytics tables (data warehouses), situations where joins are too slow even with good indexes. Trade-off: faster reads, but data updates require updating multiple copies of data — risk of inconsistency. Common in OLAP (analytics) systems, less common in OLTP (transactional) systems.

Q33: What is the ACID property in DBMS? Explain each letter.

Answer: ACID is the set of four rules that guarantee database transactions are processed reliably. Atomicity: A transaction is all-or-nothing. If you transfer money from Account A to B, both the debit and credit must succeed. If either fails, NEITHER happens. Consistency: The database moves from one valid state to another. Rules and constraints are never violated. Isolation: Multiple transactions running at the same time don't interfere with each other. Each transaction behaves as if it's the only one running. Durability: Once a transaction is committed, the changes are permanent — even if the server crashes immediately after.

Q34: What is a database transaction? How do you use BEGIN, COMMIT, and ROLLBACK?

Answer: A transaction is a group of SQL operations that are treated as one unit. Either ALL succeed (COMMIT saves them) or ALL fail (ROLLBACK undoes them). BEGIN starts the transaction. COMMIT saves all changes permanently. ROLLBACK undoes all changes since BEGIN. SAVEPOINT creates a mid-transaction checkpoint you can roll back to without undoing the entire transaction.

```
BEGIN;  -- start transaction

UPDATE accounts SET balance = balance - 500 WHERE id = 1;  -- debit
UPDATE accounts SET balance = balance + 500 WHERE id = 2;  -- credit

-- Check if both updates look correct, then save
```

```
COMMIT;

-- If something went wrong instead:
-- ROLLBACK;  -- undoes both updates

-- SAVEPOINT example
BEGIN;
INSERT INTO orders ...;
SAVEPOINT after_order;
INSERT INTO order_items ...;  -- this fails
ROLLBACK TO after_order;      -- go back to savepoint, order is kept
COMMIT;
```

Q35: What are Transaction Isolation Levels and what problems do they prevent?

Answer: Isolation levels control how much one transaction can see changes made by other concurrent transactions. Four levels (weakest to strongest): READ UNCOMMITTED — can read uncommitted data from other transactions (dirty reads). READ COMMITTED — can only read committed data (prevents dirty reads). This is the DEFAULT in PostgreSQL. REPEATABLE READ — if you read a row twice in a transaction, you get the same value both times (prevents non-repeatable reads). SERIALIZABLE — strictest level, transactions behave as if they ran one after another (prevents phantom reads). Three problems they prevent: Dirty Read (reading uncommitted data), Non-repeatable Read (same query returns different data within one transaction), Phantom Read (new rows appearing in a repeated query).

Q36: What is a deadlock in a database? How does PostgreSQL handle it?

Answer: A deadlock is a situation where two or more transactions are each waiting for the other to release a lock — they are stuck in a permanent standoff. Example: Transaction A locks Table 1, then waits for Table 2. Transaction B locks Table 2, then waits for Table 1. Neither can proceed. PostgreSQL automatically detects deadlocks and kills one of the transactions (rolling it back) to break the cycle. To prevent deadlocks: always lock tables in the same order, keep transactions short, and release locks as quickly as possible.

Q37: What is a cursor in a database and when would you use one?

Answer: A cursor is a pointer that lets you process query results one row at a time, rather than getting all rows at once. Think of it like a moving bookmark through a result set. Use cursors when: the result set is too large to load into memory all at once, you need to process each row with complex logic, you need to fetch rows in batches. In most cases, set-based SQL is faster than cursors — avoid cursors unless necessary.

```
-- PostgreSQL cursor example
DO $$
DECLARE
    emp RECORD;
    emp_cursor CURSOR FOR SELECT id, name, salary FROM employee;
BEGIN
    OPEN emp_cursor;
    LOOP
        FETCH emp_cursor INTO emp;
        EXIT WHEN NOT FOUND;
        -- Process each row
        RAISE NOTICE 'Employee: %, Salary: %', emp.name, emp.salary;
    END LOOP;
    CLOSE emp_cursor;
END;
$$;
```

Q38: What are the different types of database locks?

Answer: Locks prevent two transactions from conflicting when accessing the same data. Shared Lock (Read Lock): multiple transactions can read the same data simultaneously, but nobody can write while shared locks are held. Exclusive Lock (Write Lock): only one transaction can hold it. No other

transaction can read or write the locked resource. Row-Level Lock: locks a specific row. Reduces contention vs table-level locks. Table-Level Lock: locks the entire table. Simple but causes lots of contention. PostgreSQL uses row-level locks by default for most operations, making it very concurrent. Advisory Locks: application-level locks you manage manually.

Q39: What is MVCC (Multi-Version Concurrency Control)?

Answer: MVCC is a clever technique that lets readers and writers work at the same time without blocking each other. Instead of locking a row when you update it, the database creates a NEW version of the row. Readers see the old version; the writer works on the new version. PostgreSQL uses MVCC extensively — this is why reads don't block writes and writes don't block reads. The downside is that old row versions accumulate over time and need to be cleaned up — this is what VACUUM does in PostgreSQL.

Q40: What is the difference between optimistic and pessimistic locking?

Answer: Both strategies handle concurrent updates to the same data, but differently. Pessimistic Locking: assume conflicts WILL happen — lock the resource before reading, hold the lock until done. Prevents conflicts but reduces concurrency. Example: SELECT FOR UPDATE in SQL. Optimistic Locking: assume conflicts are RARE — don't lock anything. Instead, when you update, check if the data has changed since you read it (using a version number or timestamp). If it has changed, reject the update and retry. Better for high-concurrency read-heavy systems. Used extensively in REST APIs and ORMs.

Section 5: PostgreSQL Fundamentals (Q41–Q48) 🐘

These questions are PostgreSQL-specific. Green header = Postgres-specific question.

PostgreSQL (often called 'Postgres') is a powerful open-source database. It goes beyond standard SQL with features like JSONB, custom data types, and extensibility. These questions appear in any role that mentions PostgreSQL.

Q41: What is PostgreSQL and how is it different from MySQL?

Answer: PostgreSQL is a free, open-source Object-Relational Database Management System (ORDBMS). It was born at the University of California, Berkeley in the 1980s and is now maintained by a large global community. Key differences from MySQL: PostgreSQL is fully ACID compliant by default for all storage engines (MySQL's ACID compliance depends on the engine — InnoDB is compliant, MyISAM is not). PostgreSQL supports advanced data types like JSONB, arrays, UUID, hstore, geometric types. PostgreSQL is more standards-compliant (follows SQL standard more strictly). MySQL is traditionally faster for simple read-heavy web apps; PostgreSQL excels at complex queries, analytics, and write-heavy workloads.

```
-- PostgreSQL supports these that MySQL doesn't natively
CREATE TABLE products (
    id       SERIAL PRIMARY KEY,
    tags     TEXT[],              -- arrays
    metadata JSONB,              -- binary JSON
    location POINT               -- geometric type
);
```

Q42: What are the key data types in PostgreSQL that make it special?

Answer: Beyond standard SQL types, PostgreSQL offers: SERIAL/BIGSERIAL — auto-incrementing integers (like AUTO_INCREMENT). TEXT — unlimited length string (no need to guess VARCHAR length). BOOLEAN — true/false (MySQL uses TINYINT). UUID — globally unique identifier. JSONB — binary JSON, indexable and faster than JSON. JSON — text JSON, preserves whitespace. ARRAY — columns can store arrays of any type. NUMERIC(precision, scale) — exact decimal for financial data (never use FLOAT for money!). TIMESTAMP WITH TIME ZONE (TIMESTAMPTZ) — stores time with timezone info. DATE, TIME — date and time only.

Q43: What is MVCC in PostgreSQL and how does it work?

Answer: MVCC (Multi-Version Concurrency Control) is PostgreSQL's way of letting many users read and write at the same time without blocking each other. How it works: when you UPDATE a row, PostgreSQL does NOT overwrite it. Instead it marks the old row as 'expired' (using a hidden xmax column) and creates a new row version. Each transaction has a transaction ID (xid). When you read data, PostgreSQL shows you only rows that were committed before your transaction started — your own snapshot of the database. This means: readers never block writers, writers never block readers. The downside: old row versions accumulate over time and waste space — this is why VACUUM exists.

Q44: What is VACUUM in PostgreSQL and why is it important?

Answer: VACUUM is PostgreSQL's housekeeping process. Because MVCC keeps old row versions around (dead tuples), space fills up over time. VACUUM removes dead tuples and reclaims space. AUTOVACUUM is the background process that runs VACUUM automatically — you should never need to run it manually in normal operation. VACUUM ANALYZE also updates statistics that the query planner uses. VACUUM FULL rebuilds the entire table and reclaims disk space but locks the table (use sparingly). Signs you need more aggressive vacuuming: bloated tables, slow queries, high dead tuple count (visible in pg_stat_user_tables).

```
-- Check dead tuples in tables
SELECT relname, n_live_tup, n_dead_tup,
       last_autovacuum, last_autoanalyze
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC;

-- Manual vacuum (if autovacuum is lagging)
VACUUM ANALYZE my_large_table;
```

Q45: What is the difference between JSON and JSONB in PostgreSQL?

Answer: Both store JSON data but in different ways. JSON stores the JSON as plain text — it preserves whitespace, key order, and duplicate keys. It is slower to query because it must parse the text every time. JSONB stores JSON in a decomposed binary format — whitespace is removed, keys are sorted, and duplicate keys are not kept. JSONB can be indexed (using GIN index) and queried much faster. Rule of thumb: always use JSONB unless you specifically need to preserve the exact original JSON text format (rare).

```
-- JSONB with GIN index for fast querying
CREATE TABLE products (
    id       SERIAL PRIMARY KEY,
    metadata JSONB
);

CREATE INDEX idx_metadata_gin ON products USING GIN(metadata);

-- Query inside JSONB
SELECT * FROM products
WHERE metadata @> '{"brand": "Nike"}';  -- contains operator

-- Extract a field
SELECT metadata->>'brand' AS brand FROM products;

-- Check if key exists
SELECT * FROM products WHERE metadata ? 'discount';
```

Q46: What is SERIAL in PostgreSQL and how does it work? What is IDENTITY?

Answer: SERIAL is a shorthand for creating an auto-incrementing integer column backed by a sequence. When you insert a row without specifying the id, PostgreSQL automatically assigns the next sequence value. BIGSERIAL is the same but for very large numbers. IDENTITY (PostgreSQL 10+) is the modern SQL standard alternative to SERIAL. Use GENERATED ALWAYS AS IDENTITY (prevents manual value insertion) or GENERATED BY DEFAULT AS IDENTITY (allows manual override). New code should prefer IDENTITY over SERIAL.

```
-- SERIAL (older, still widely used)
CREATE TABLE orders (id SERIAL PRIMARY KEY, total NUMERIC);

-- IDENTITY (modern, SQL standard)
CREATE TABLE orders (
    id    INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    total NUMERIC
);

-- See current sequence value
SELECT last_value FROM orders_id_seq;
```

Q47: What is a schema in PostgreSQL and how is it different from a database?

Answer: In PostgreSQL, a database is the top-level container. Inside a database, a schema is a namespace that holds tables, views, indexes, sequences, and other objects. Think of it as a folder inside the database. By default, PostgreSQL uses the 'public' schema. Benefits of schemas: organise objects by purpose (app schema, audit schema, staging schema), allow multiple users to have their own namespace without conflicts, multi-tenant apps can give each customer their own schema. You can switch schemas using search_path.

```
-- Create a schema
CREATE SCHEMA sales;

-- Create table inside a specific schema
CREATE TABLE sales.orders (id SERIAL PRIMARY KEY, total NUMERIC);

-- Set default schema for session
SET search_path TO sales, public;

-- Show all schemas
SELECT schema_name FROM information_schema.schemata;
```

Q48: How do you backup and restore a PostgreSQL database?

Answer: Logical Backup with pg_dump: creates a SQL file that can recreate the database. Easy to move between versions. pg_dumpall backs up all databases. pg_restore restores a pg_dump backup. Physical Backup with pg_basebackup: copies the actual data files. Used for replication and disaster recovery. Point-in-Time Recovery (PITR): combine physical backup with WAL archiving to restore to any point in time. Always test your backups on a staging server!

```
-- Logical backup (single database, SQL format)
pg_dump -U postgres -d mydb -f mydb_backup.sql

-- Logical backup (custom compressed format — recommended)
pg_dump -U postgres -F c -d mydb -f mydb_backup.dump

-- Restore SQL format
psql -U postgres -d mydb < mydb_backup.sql

-- Restore custom format
pg_restore -U postgres -d mydb mydb_backup.dump

-- All databases backup
pg_dumpall -U postgres > all_databases.sql

-- Physical backup (for replication/HA)
pg_basebackup -U replication_user -D /backup/pg_base -Fp -Xs -P
```

Section 6: PostgreSQL Indexing & Performance (Q49–Q55) 🐘

Performance tuning is one of the most valuable skills for a PostgreSQL developer. These questions are commonly asked for mid-to-senior roles.

Q49: What types of indexes does PostgreSQL support? When do you use each?

Answer: PostgreSQL has more index types than most databases. B-tree (default): balanced tree structure, works for most queries — equality (=), ranges (<, >), sorting (ORDER BY), LIKE 'prefix%'. Hash: only for exact equality (=) — faster than B-tree for equality but cannot be used for ranges. GIN (Generalized Inverted Index): for multi-value data — arrays, JSONB, full-text search. Best for 'contains' queries. GiST (Generalized Search Tree): for geometric data, range types, and full-text search. SP-GiST: for space-partitioned data (IP addresses, points). BRIN (Block Range Index): for very large tables where data is naturally ordered (like time-series) — tiny size, very fast to create.

```
-- B-tree (default, most common)
CREATE INDEX idx_email ON users(email);

-- GIN for JSONB
CREATE INDEX idx_meta ON products USING GIN(metadata);

-- GIN for full-text search
CREATE INDEX idx_fts ON articles USING GIN(to_tsvector('english', body));

-- BRIN for large time-series table (tiny index, good for ordered data)
CREATE INDEX idx_log_ts ON logs USING BRIN(created_at);

-- Partial index (index only active users -- smaller, faster)
CREATE INDEX idx_active_users ON users(email) WHERE active = true;

-- Expression index
CREATE INDEX idx_lower_email ON users(LOWER(email));
```

Q50: What is EXPLAIN and EXPLAIN ANALYZE? How do you read a query plan?

Answer: EXPLAIN shows the query execution plan — how PostgreSQL PLANS to run your query (estimated cost). EXPLAIN ANALYZE actually RUNS the query and shows real execution time and row counts. Key things to look for in a query plan: Seq Scan = full table scan (slow on large tables — usually needs an index). Index Scan = using an index (good). Index Only Scan = even better — all needed data is in the index itself. Nested Loop, Hash Join, Merge Join = different ways to join tables. High 'rows' estimates that differ wildly from actual rows = stale statistics (run ANALYZE). High actual time on a node = that's your bottleneck.

```
-- Just show the plan (doesn't actually run)
EXPLAIN SELECT * FROM orders WHERE customer_id = 42;

-- Run it and show real timings
EXPLAIN ANALYZE SELECT * FROM orders WHERE customer_id = 42;

-- Verbose output with buffer info
EXPLAIN (ANALYZE, BUFFERS, VERBOSE, FORMAT TEXT)
SELECT o.id, c.name
FROM orders o JOIN customers c ON o.customer_id = c.id
WHERE o.total > 1000;

-- Example output reading:
-- Seq Scan on orders  (cost=0.00..12500.00 rows=1000 width=50)
--   Filter: (total > 1000)
--   Rows Removed by Filter: 49000
-- ^ 49000 rows scanned but discarded -> needs an index on 'total'
```

Q51: What is pg_stat_statements and how do you use it for performance tuning?

Answer: pg_stat_statements is a PostgreSQL extension that tracks statistics for every SQL query that has been run: total execution time, call count, average time, rows returned, and more. It is the single most useful tool for finding slow queries. Enable it by adding pg_stat_statements to shared_preload_libraries in postgresql.conf, then CREATE EXTENSION pg_stat_statements;.

```
-- Enable (add to postgresql.conf)
-- shared_preload_libraries = 'pg_stat_statements'

-- Enable extension in database
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

-- Find the top 10 slowest queries (by total execution time)
SELECT query,
       calls,
       total_exec_time / 1000 AS total_sec,
       mean_exec_time          AS avg_ms,
       rows
FROM   pg_stat_statements
ORDER  BY total_exec_time DESC
LIMIT  10;

-- Find most frequently called queries
SELECT query, calls, mean_exec_time
FROM pg_stat_statements
ORDER BY calls DESC LIMIT 10;
```

Q52: What is table partitioning in PostgreSQL and when should you use it?

Answer: Partitioning splits one large table into smaller physical pieces called partitions, while still appearing as one table to queries. PostgreSQL supports three partitioning strategies. RANGE: split by range of values — great for dates/times (one partition per month/year). LIST: split by specific values — e.g., one partition per country. HASH: split by hash of a column — good for evenly distributing data with no natural range. Benefits: queries that filter on the partition key only scan the relevant partition (partition pruning), maintenance is easier (archive old partitions by detaching them), VACUUM and index operations can run per-partition.

```
-- RANGE partitioning by year
CREATE TABLE sales (
    id         SERIAL,
    sale_date  DATE NOT NULL,
    amount     NUMERIC
) PARTITION BY RANGE (sale_date);

-- Create partitions for each year
CREATE TABLE sales_2023
    PARTITION OF sales
    FOR VALUES FROM ('2023-01-01') TO ('2024-01-01');

CREATE TABLE sales_2024
    PARTITION OF sales
    FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');

-- PostgreSQL automatically routes inserts to the right partition
-- Queries with WHERE sale_date = '2024-06-15' only scan sales_2024
```

Q53: What is connection pooling in PostgreSQL and why is it needed?

Answer: PostgreSQL creates a new OS process for every connection — each connection uses about 10MB of RAM and takes ~5ms to establish. If your app opens a new DB connection for every request, you'll quickly hit limits. Connection pooling keeps a pool of pre-opened connections and reuses them across requests. PgBouncer is the most popular connection pooler for PostgreSQL — it sits between your app and PostgreSQL and multiplexes many app connections into fewer server connections. Modes: session pooling, transaction pooling (most common — reassigns connection per transaction), statement pooling.

```
-- Configure max connections in postgresql.conf
-- max_connections = 200  (PostgreSQL default)

-- PgBouncer sits between app and PostgreSQL
-- App connects to PgBouncer port 5432
-- PgBouncer manages a pool of real PostgreSQL connections

-- Check current connections
SELECT count(*) FROM pg_stat_activity WHERE state = 'active';

-- See all active queries
SELECT pid, usename, application_name, state, query
FROM pg_stat_activity
WHERE state != 'idle'
ORDER BY query_start;
```

Q54: What important PostgreSQL configuration parameters affect performance?

Answer: shared_buffers: memory PostgreSQL uses for caching data. Set to 25% of total RAM. work_mem: memory per sort/hash operation per query. Higher = fewer disk spills. Set carefully for concurrent workloads (each query can use this much). maintenance_work_mem: memory for VACUUM, index creation. Can be set higher. effective_cache_size: hint to query planner about how much OS cache is available. Set to ~75% of total RAM. max_connections: maximum concurrent connections. Keep low, use PgBouncer instead. checkpoint_completion_target: spread checkpoint I/O over time (set to 0.9). wal_buffers: WAL write buffer size.

```
-- Key settings in postgresql.conf
shared_buffers = 4GB              -- 25% of 16GB RAM
work_mem = 64MB                   -- per sort/hash operation
maintenance_work_mem = 512MB      -- for VACUUM, REINDEX
effective_cache_size = 12GB       -- 75% of 16GB RAM
checkpoint_completion_target = 0.9
max_connections = 100             -- use PgBouncer for more
wal_buffers = 16MB

-- After changing postgresql.conf, reload without restart
SELECT pg_reload_conf();
```

Q55: How do you find and kill long-running queries in PostgreSQL?

Answer: Long-running queries can block other queries and degrade performance. PostgreSQL's pg_stat_activity system view shows all current activity. You can terminate queries using pg_cancel_backend (graceful, sends cancellation signal) or pg_terminate_backend (forceful, kills the connection).

```
-- Find queries running more than 5 minutes
SELECT pid, usename, state, query_start,
       now() - query_start AS duration, query
FROM   pg_stat_activity
WHERE  state != 'idle'
  AND  now() - query_start > INTERVAL '5 minutes'
ORDER  BY duration DESC;

-- Gracefully cancel a query (leave connection open)
SELECT pg_cancel_backend(12345);  -- 12345 = pid from above

-- Forcefully kill a connection
SELECT pg_terminate_backend(12345);

-- Kill all idle connections older than 10 minutes
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE state = 'idle'
AND now() - state_change > INTERVAL '10 minutes';
```

Section 7: PostgreSQL Advanced Features (Q56–Q65) 🐘

These questions come up for senior developer and DBA roles — they cover the powerful features that make PostgreSQL a production-grade database.

Q56: What is Write-Ahead Logging (WAL) in PostgreSQL?

Answer: WAL is PostgreSQL's crash recovery mechanism. The rule is simple: before writing any change to the actual data files on disk, PostgreSQL first writes a record of that change to the WAL log (stored in the pg_wal directory). This way, if the server crashes mid-operation, PostgreSQL can replay the WAL log on restart to recover all committed changes. WAL enables two more powerful features: Replication (standby servers read the WAL stream from the primary), and Point-in-Time Recovery (PITR — restore the database to any moment in the past by replaying WAL up to that timestamp).

Q57: What is replication in PostgreSQL? Explain streaming replication.

Answer: Replication keeps a copy of the database on one or more standby servers. This provides high availability and read scalability. Streaming Replication: the standby server continuously reads the WAL stream from the primary server and applies it in real time. The standby is almost always up-to-date. The standby can be configured as hot standby (readable) or warm standby (not readable). Synchronous Replication: primary waits for at least one standby to confirm receiving WAL before committing — zero data loss, but adds latency. Asynchronous Replication: primary does not wait — faster, but risk of losing a few transactions if primary crashes. Logical Replication: replicates changes at the row level, selectively by table, useful for version upgrades and partial replication.

```
-- Enable replication in postgresql.conf (primary server)
wal_level = replica
max_wal_senders = 10
hot_standby = on

-- Create replication user
CREATE USER replicator REPLICATION LOGIN PASSWORD 'secret';

-- Start streaming replication on standby
-- pg_basebackup -h primary_host -U replicator -D /data/pgstandby
```

Q58: What is Row-Level Security (RLS) in PostgreSQL?

Answer: RLS allows you to control which rows a user can see or modify based on a security policy. It's like a WHERE clause that is automatically added to every query for that table. Perfect for multi-tenant applications where each user should only see their own data. Enable with ALTER TABLE ... ENABLE ROW LEVEL SECURITY, then define policies with CREATE POLICY.

```
-- Enable RLS on the orders table
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;

-- Policy: users can only see their own orders
CREATE POLICY user_orders_policy
    ON orders
    USING (customer_id = current_user_id());

-- Policy: admins can see everything
CREATE POLICY admin_policy
    ON orders
    TO admin_role
    USING (true);

-- Now when a normal user runs:
-- SELECT * FROM orders;
-- PostgreSQL automatically adds: WHERE customer_id = current_user_id()
```

Q59: What is full-text search in PostgreSQL and how does it work?

Answer: PostgreSQL has built-in full-text search — no need for Elasticsearch for basic search needs. Key concepts: tsvector is a processed, searchable representation of a text document. tsquery is the search query. The @@ operator performs the match. to_tsvector() converts text to a tsvector. to_tsquery() converts a search term to a tsquery. Create a GIN index on the tsvector column for fast search. PostgreSQL supports stemming, stop words, and ranking by relevance.

```
-- Add a tsvector column for fast full-text search
ALTER TABLE articles ADD COLUMN search_vector TSVECTOR;

-- Update it (or use a trigger to keep it automatic)
UPDATE articles
SET search_vector = to_tsvector('english', title || ' ' || body);

-- Create GIN index
CREATE INDEX idx_fts ON articles USING GIN(search_vector);

-- Search for articles containing 'database' and 'performance'
SELECT title FROM articles
WHERE search_vector @@ to_tsquery('english', 'database & performance');

-- Ranked search results
SELECT title, ts_rank(search_vector,
    to_tsquery('database & performance')) AS rank
FROM articles
WHERE search_vector @@ to_tsquery('database & performance')
ORDER BY rank DESC;
```

Q60: What are PostgreSQL sequences and how do you work with them?

Answer: A sequence is a database object that generates a series of unique numbers. SERIAL and BIGSERIAL columns use sequences internally. You can also create sequences manually for custom needs. Key functions: nextval('seq_name') — get the next value (and advance the sequence). currval('seq_name') — get the last value returned in this session. setval('seq_name', value) — manually set the current value. Sequences are not rolled back with ROLLBACK — if a transaction fails, the sequence number is lost (gaps in IDs are normal and expected).

```
-- Create a sequence manually
CREATE SEQUENCE order_number_seq
    START WITH 1000
    INCREMENT BY 1
    NO MAXVALUE;

-- Use it
INSERT INTO orders (order_number, total)
VALUES (nextval('order_number_seq'), 250.00);

-- Get current value in this session
SELECT currval('order_number_seq');

-- Reset a sequence (e.g. after bulk delete)
SELECT setval('orders_id_seq', (SELECT MAX(id) FROM orders));
```

Q61: What are CTEs (Common Table Expressions) in PostgreSQL and what makes them special?

Answer: CTEs are temporary, named result sets defined using the WITH keyword — they exist only for the duration of one query. They make complex queries readable by naming intermediate steps. PostgreSQL supports three types: Regular CTE: a named subquery you reuse. Data-modifying CTE: you can use WITH with INSERT, UPDATE, DELETE — changes rows and uses the results. Recursive CTE: the CTE calls itself — perfect for hierarchical data (employee trees, category paths, graph traversal). Important note: in older PostgreSQL versions, CTEs were 'optimization fences' — the planner could not inline them. In PostgreSQL 12+, non-recursive CTEs are inlined by default, improving performance.

```
-- Data-modifying CTE: move deleted rows to an archive table
WITH deleted AS (
    DELETE FROM orders
    WHERE order_date < '2020-01-01'
    RETURNING *
)
INSERT INTO orders_archive SELECT * FROM deleted;

-- Recursive CTE: build category path
WITH RECURSIVE category_path AS (
    SELECT id, name, parent_id, name::TEXT AS path
    FROM categories WHERE parent_id IS NULL
    UNION ALL
    SELECT c.id, c.name, c.parent_id,
           cp.path || ' > ' || c.name
    FROM categories c
    JOIN category_path cp ON c.parent_id = cp.id
)
SELECT id, path FROM category_path ORDER BY path;
```

Q62: What are PostgreSQL extensions? Name some commonly used ones.

Answer: PostgreSQL has a plugin system called extensions that add new functionality without modifying the core database. Common extensions: pg_stat_statements — query performance tracking. uuid-ossp — generates UUIDs (universally unique identifiers). pgcrypto — encryption functions. PostGIS — geographic/spatial data queries (used by mapping applications). hstore — key-value storage in a single column. pg_trgm — trigram-based similarity search (fuzzy matching). timescaledb — time-series data extension. Install with CREATE EXTENSION.

```
-- Install commonly used extensions
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- Use UUID as primary key
CREATE TABLE users (
    id   UUID DEFAULT uuid_generate_v4() PRIMARY KEY,
    name TEXT
);

-- Trigram similarity search (fuzzy name matching)
SELECT name FROM products
WHERE name % 'Postgress'  -- matches 'PostgreSQL' despite typo
ORDER BY similarity(name, 'Postgress') DESC;
```

Q63: How do you implement table inheritance in PostgreSQL?

Answer: PostgreSQL is an Object-Relational database — it supports table inheritance. A child table inherits all columns from a parent table and can add its own columns. Queries on the parent automatically include rows from child tables. Use cases: partitioning (before declarative partitioning existed), modelling object hierarchies. Note: foreign keys do NOT automatically reference child tables. Modern partitioning (PARTITION BY) is usually preferred over inheritance for performance reasons.

```
-- Parent table
CREATE TABLE vehicles (id SERIAL, make TEXT, model TEXT, year INT);

-- Child tables inherit all parent columns
CREATE TABLE cars   (doors INT) INHERITS (vehicles);
CREATE TABLE trucks (payload_kg NUMERIC) INHERITS (vehicles);

-- Insert into child
INSERT INTO cars (make, model, year, doors)
VALUES ('Toyota', 'Corolla', 2023, 4);
```

```
-- Query parent includes all children
SELECT * FROM vehicles;  -- includes cars AND trucks

-- Query only one table
SELECT * FROM ONLY vehicles;  -- excludes children
```

Q64: What is logical replication in PostgreSQL and how is it different from streaming replication?

Answer: Streaming replication replicates the entire database at the physical WAL level — it sends every byte change to the standby. The standby must run the exact same PostgreSQL version and architecture. Logical replication replicates at the logical (row) level — you can choose specific tables to replicate, replicate between different PostgreSQL versions, replicate to a completely different database system, and filter what gets replicated. Use cases: database migrations with zero downtime, replicating select tables to an analytics replica, upgrading major PostgreSQL versions with minimal downtime.

```
-- On source database: create publication
CREATE PUBLICATION my_pub FOR TABLE orders, customers;

-- On destination database: create subscription
CREATE SUBSCRIPTION my_sub
    CONNECTION 'host=source_host dbname=mydb user=replicator'
    PUBLICATION my_pub;

-- Check replication status
SELECT * FROM pg_stat_replication;
SELECT * FROM pg_stat_subscription;
```

Q65: How do you handle schema migrations in PostgreSQL in production with minimal downtime?

Answer: Zero-downtime migrations are tricky because some DDL operations lock the table. Safe patterns: Adding a nullable column: safe, no lock needed. Adding a NOT NULL column: risky — add nullable first, backfill, then add constraint. ALTER TABLE ... ADD COLUMN GENERATED is safer in newer versions. Creating an index: use CREATE INDEX CONCURRENTLY — builds index without locking writes. Renaming a column: rename is instant but applications using the old name will break — use multi-step (add new column, backfill, update app, drop old). For large tables: never run bare ALTER TABLE on millions of rows — break it into batches. Tools like pgquarrel and Flyway/Liquibase help manage migrations systematically.

```
-- Safe: add nullable column
ALTER TABLE users ADD COLUMN phone TEXT;

-- Safe index creation: CONCURRENTLY = no write locks
CREATE INDEX CONCURRENTLY idx_users_phone ON users(phone);

-- Safe NOT NULL pattern (3 steps):
-- Step 1: Add nullable column
ALTER TABLE users ADD COLUMN tier TEXT;
-- Step 2: Backfill in batches
UPDATE users SET tier = 'standard' WHERE id BETWEEN 1 AND 100000;
-- ... repeat for all batches ...
-- Step 3: Add constraint ONLY after all rows are filled
ALTER TABLE users ALTER COLUMN tier SET NOT NULL;

-- Check for locks before running migrations
SELECT pid, mode, relation::regclass, granted
FROM pg_locks l JOIN pg_class c ON l.relation = c.oid
WHERE NOT granted;
```

Quick Reference: DBMS vs PostgreSQL Concepts

Concept	General DBMS/SQL	PostgreSQL Specifics
Auto-increment	AUTO_INCREMENT (MySQL)	SERIAL, BIGSERIAL, or IDENTITY (recommended)
JSON storage	Not always supported	JSON (text) and JSONB (binary, indexable) — always use JSONB
Full-text search	Not built-in (needs tool)	Built-in: tsvector + tsquery + GIN index
Index types	B-tree, Clustered, Non-clustered	B-tree, Hash, GIN, GiST, SP-GiST, BRIN, Partial, Expression
Concurrency model	Lock-based (most DBs)	MVCC — readers never block writers
Vacuum/Cleanup	Automatic (hidden)	Must VACUUM dead tuples explicitly (autovacuum runs automatically)
Replication	Vendor-specific	Streaming (physical), Logical (row-level, selective)
Crash recovery	Redo logs	WAL (Write-Ahead Log) in pg_wal directory
Query analysis	EXPLAIN	EXPLAIN ANALYZE + pg_stat_statements for real stats
Partitioning	Manual or vendor	RANGE, LIST, HASH partitioning — declarative since v10
Connection limit	Session-based	Hard limit per server — use PgBouncer pooler
Row-level security	Application-managed	Native RLS with CREATE POLICY
Default isolation	Varies	READ COMMITTED (configurable per transaction)
Backup tool	mysqldump etc.	pg_dump (logical), pg_basebackup (physical)
Schema namespace	Database scoped	Schema inside database (public schema by default)

⚠ Always use JSONB over JSON in PostgreSQL unless you specifically need exact text preservation.
⚠ SERIAL still works but GENERATED AS IDENTITY is the modern SQL standard replacement.
⚠ CREATE INDEX CONCURRENTLY is essential for production — never create indexes without it on live tables.
⚠ autovacuum should never be disabled — it keeps MVCC dead tuples cleaned up.
⚠ READ COMMITTED is PostgreSQL's default isolation level — use SERIALIZABLE for strict financial operations.